Sistemas de Coordenadas (CB104) · Agrimensura · FIUBA

Instructivo para docentes

Cómo editar y agregar material, y cómo se publica

Material del trabajo final de la materia · Gustavo Varela · octubre de 2026

La regla única

Todo lo que se ve en el sitio y en el PDF sale del repositorio. Se edita la *fuente* (un archivo de texto), se corre una verificación, se regenera lo derivado y se publica con un `git push` a la rama `main`. El sitio se reconstruye solo en unos minutos; nadie sube archivos a mano.
Este instructivo recorre, pieza por pieza, los dos caminos: **editar** lo que ya existe y **agregar** algo nuevo. Al final está la lista de verificación previa a publicar (sección 8) y el resumen de comandos (sección 9). La instalación de Git y `uv` y el uso básico del repositorio están en el *Instructivo para estudiantes*; acá se da por hecho.

Contenido

1. Mapa de las piezas . . . 1
2. Antes de empezar: una sola vez por computadora . . . 2
3. Unidades interactivas . . . 2
4. Banco de preguntas y autoevaluación . . . 4
5. El apunte . . . 5
6. Fichas de capítulo y mapa de conceptos . . . 6
7. Animaciones y laboratorio CAD . . . 6
8. Verificar y publicar . . . 7
9. Resumen: qué comando para cada tarea . . . 8

1. Mapa de las piezas

Cada pieza del material tiene una fuente que se edita, un derivado que se genera y un comando que lo genera. Editar el derivado a mano no sirve: la próxima generación lo pisa, y en varios casos la publicación falla si el derivado no coincide con su fuente.

Pieza	Fuente (se edita)	Derivado (no se edita)	Generador
Unidad interactiva	`unidades/<id>.py`	`site/unidades/<id>/`	`tools/build.py`
Biblioteca común	`unidades/cb104.py`	(se inyecta en cada unidad)	`tools/build.py`
Hilo conductor y portada	`mapa/_fragmentos/unidades.yaml`, `HILO_CONDUCTOR` en `cb104.py`, `tools/enlaces_portada.yaml`	`mapa/mapa.yaml`, `site/index.html`	`tools/merge_mapa.py`, `tools/build.py`
Mapa de conceptos	`mapa/_fragmentos/bloque-*.yaml`, `mapa/CONCEPTOS.md`	`mapa/mapa.yaml`, `site/mapa.html`	`tools/merge_mapa.py`, `tools/build.py`
Fichas de capítulo	`clases/NN-tema.md`	`site/capitulos/`	`tools/build.py`

Pieza	Fuente (se edita)	Derivado (no se edita)	Generador
Banco de preguntas	`unidades/_banco/bloque_N.py`	`unidades/banco_autoevaluacion.py`, `site/autoevaluacion.html`, XML de Moodle	`tools/merge_banco.py`, `tools/build.py`
Apunte	`apunte/contenido/NN-nombre.tex`, `apunte-formato.sty`	`apunte/Apunte_Sistemas_de_Coordenadas.pdf`, `paginas_esperadas.txt`, `numeracion.json`	`scripts/compilar_pdf.sh`, `apunte/scripts/numeracion.py`
Problemas por capítulo	`tools/problemas/gen.py`	`apunte/contenido/problemas/pNN.tex`, `ap6-respuestas.tex`	`tools/problemas/gen.py`
Animaciones	`tools/animaciones/anim_NN_*.py`	`animaciones/*.mp4`, `*.gif`	`tools/gen_animaciones.py`
Laboratorio CAD	`tools/cad/generar_dxf.py`, `cad/ejercicios.yaml`, `cad/index.md`	`cad/dxf/**/*.dxf`	`tools/cad/generar_dxf.py`

Dos archivos que sí se editan a mano aunque parezcan generados

`CURSO_INTERACTIVO.md` (la descripción de las veinte simulaciones) y `README.md` no tienen generador. Si cambia una unidad, hay que actualizarlos a mano.

2. Antes de empezar: una sola vez por computadora

1. Instalar Git y `uv`, clonar el repositorio y correr `uv sync` (pasos 1 a 3 del instructivo para estudiantes).
2. Activar los ganchos de verificación del repositorio, que protegen el PDF del apunte en cada commit:
   `git config core.hooksPath .githooks`
3. Comprobar que la batería completa de pruebas pasa en limpio antes de tocar nada; así cualquier falla posterior es propia:

```
uv run tools/merge_mapa.py --check
uv run python tools/merge_banco.py --check
uv run pytest -q
uv run python apunte/scripts/estilo.py --todo --estricto
```

Para recompilar el apunte hace falta además TeX Live con LuaLaTeX, `latexmk`, `pdfinfo` y las fuentes Linux Libertine O y DejaVu. Para las animaciones, `ffmpeg`. Para todo lo demás alcanza con `uv`.

3. Unidades interactivas

3.1. Editar una unidad existente

Cada unidad es un cuaderno de marimo en `unidades/<id>.py`. Los `<id>` están en la tabla del instructivo para estudiantes (por ejemplo `08_cambio_de_base`). Para abrirla con las celdas a la vista:
`uv run marimo edit unidades/08_cambio_de_base.py`

Se abre en el navegador; cada cambio en una celda se ve al momento, y al guardar se escribe el `.py`. Si preferís ensayar sin tocar el original, trabajá sobre una copia con guion bajo inicial (`unidades/_ensayo_08.py`): el sitio nunca la exporta.

Tres reglas para no romper la exportación

1. No borrar el encabezado `## /// script ... ## ///` del principio del archivo ni la celda `import cb104 as lib ## cb104:inline`, que tiene que quedar exactamente así, en una celda propia. El generador los usa para declarar las dependencias e inyectar la biblioteca común en la página exportada.
2. Las variables que viven en una sola celda llevan guion bajo (`_fig`, `_panel`, `_res`). Marimo exige nombres únicos entre celdas.
3. Las funciones de ayuda nuevas van a `unidades/cb104.py`, con su `assert` en el bloque `__main__`, no a la lección. En `cb104.py` solo se importa lo que existe en el navegador: hoy NumPy y Plotly.

Verificar antes de dar por buena la edición:

```
uv run marimo check --select MW unidades/08_cambio_de_base.py
uv run python unidades/08_cambio_de_base.py
uv run python tools/build.py --only 08_cambio_de_base
uv run python -m http.server -d site 8765
```

El primero tiene que terminar sin avisos; el segundo corre todas las celdas con sus `assert`; el tercero exporta solo esa unidad; el cuarto la sirve en `http://localhost:8765/unidades/08_cambio_de_base/`. Si se tocó `cb104.py`, hay que exportar todas las unidades (`tools/build.py` sin `--only`), porque la biblioteca se inyecta en cada una.

Detalles que ya hicieron perder tiempo: las etiquetas de `mo.ui.radio` y `mo.ui.dropdown` no pasan por KaTeX, así que los vectores en las opciones se escriben con `lib.vec_plano`, no con `$...$`; una barra vertical dentro de una fórmula en una tabla Markdown parte la celda, se escribe `\lvert ... \rvert` ; en los textos `r"""..."""` los escapes van simples.

3.2. Agregar una unidad nueva

El molde es `unidades/PLANTILLA_leccion.py`: ya trae el encabezado de dependencias, la celda del import, las seis secciones (0 a 5) y el anexo. Lo que está entre corchetes se reemplaza; las notas `PLANTILLA:` se borran.

1. **Copiar la plantilla** y completarla:
   ```
   cp unidades/PLANTILLA_leccion.py unidades/NN_nombre.py
   uv run marimo edit unidades/NN_nombre.py
   ```
   Hay que cambiar el número y el título del encabezado, los datos de la predicción (punto, matriz, opciones y sus explicaciones), la parcela de la figura, los ejercicios (a) a (d), el identificador en `lib.preparar_autoevaluacion(mo, "<id>")` y los enlaces del anexo.
2. **Darla de alta en el hilo conductor**, en `mapa/_fragmentos/unidades.yaml`. La posición en la lista es la posición en la portada. Los campos son estos y solo estos:
   ```
   - id: NN_nombre                      # unico; carpeta site/unidades/<id>/
     archivo: unidades/NN_nombre.py
     titulo: "Titulo de la leccion"
     estado: piloto                     # piloto | migrada se exportan; legacy |
     ↪ pendiente no
     clases: ["NN"]                     # ids de ficha que cubre, como strings
     pregunta: "La pregunta de campo, en una sola linea"
     gancho_agrimensura: "Situacion concreta de campo, en una sola linea"
     nivel: troncal                     # troncal | complementario | avanzado
   ```
3. **Apuntar los conceptos a la lección**: en el `mapa/_fragmentos/bloque-*.yaml` de su ficha, poner `unidad: NN_nombre` en cada concepto que la lección desarrolla. Si el concepto es nuevo, antes agregarlo a `mapa/CONCEPTOS.md`.
4. **Verificar**:
   ```
   uv run tools/merge_mapa.py
   ```

```
uv run marimo check --select MW unidades/NN_nombre.py
uv run python unidades/NN_nombre.py
```

5. **Exportar y mirar**: `uv run python tools/build.py --only NN_nombre`, servir con `http.server` y abrir la unidad. Commit de la lección, los fragmentos y el `mapa/mapa.yaml` regenerado.

Lo que una unidad nueva arrastra y ningún README dice

El material está cerrado en veinte unidades y varias piezas lo dan por hecho. Agregar la vigesimoprimera obliga a tocar, además de lo anterior:

- `HILO_CONDUCTOR` en `unidades/cb104.py`: una entrada con los campos `titulo`, `venimos_de`, `ladrillo_de_hoy`, `vamos_hacia`, `anterior` y `siguiente`, en el mismo orden que el yaml, y retocar `siguiente` y `anterior` de las vecinas.
- El `assert len(_ids_yaml) == 20` del mismo archivo.
- Una entrada en `tools/enlaces_portada.yaml` con una a tres etiquetas del apunte; sin ella el generador del sitio falla.
- Seis preguntas en el banco (sección 4); `merge_banco.py` las exige.
- El exportador de Moodle supone veinte unidades; mientras falten preguntas se corre con `--parcial`.

Reemplazar una unidad existente por otra es mucho más barato que agregar una.

4. Banco de preguntas y autoevaluación

La fuente son los archivos `unidades/_banco/bloque_1.py` a `bloque_4.py`, uno por grupo de unidades, con un diccionario `BANCO = {"<id>": [...]}`. Cada unidad tiene seis preguntas: dos elementales, dos medias y dos avanzadas. Una pregunta tiene esta forma:

```
{
    "id": "00_m2",                # <unidad corta>_<e|m|a><1|2>
    "nivel": "medio",             # elemental | medio | avanzado
    "enunciado": r"... con formulas entre $...$ ...",
    "opciones": [                 # exactamente cuatro, una sola correcta
        {"id": "a", "texto": r"...", "correcta": True,  "explicacion": r"..."},
        {"id": "b", "texto": r"...", "correcta": False, "explicacion": r"..."},
        {"id": "c", "texto": r"...", "correcta": False, "explicacion": r"..."},
        {"id": "d", "texto": r"...", "correcta": False, "explicacion": r"..."},
    ],
    "rubrica": "opcional: que evalua y de que ladrillo viene",
},
```

Cada opción, correcta o no, lleva su explicación: es lo que el alumno ve al fallar. Para **editar** una pregunta se cambia el texto en el bloque; para **agregar** o reemplazar, se respeta el esquema y el cupo de seis por unidad. Después, siempre:

```
uv run python tools/merge_banco.py      # valida y regenera el banco
uv run python tools/build.py            # autoevaluacion.html y XML de Moodle
```

y se commitean el bloque editado, el `banco_autoevaluacion.py` regenerado y el XML de `moodle/`.

La publicación no revisa el banco

El flujo de publicación no corre `merge_banco.py`. Si se edita un bloque y no se commitea el archivo regenerado, el sitio publica el banco viejo sin ningún aviso. El `--check` del paso previo a publicar (sección 8) lo detecta.

Para llevarlo a Moodle: el archivo es `moodle/moodle_banco_120_preguntas_cb104.xml`. En el curso, «Más → Banco de preguntas → Importar», formato «Moodle XML». Crea una categoría por unidad y, adentro, una por nivel. El detalle está en `moodle/README.md`.

5. El apunte

5.1. Editar un capítulo

El maestro es `apunte/Apunte_Sistemas_de_Coordenadas.tex`; cada capítulo es un archivo `apunte/contenido/NN-nombre.tex` que el maestro incluye con `\input` . Todo el formato (cajas, colores, macros) vive en `apunte/apunte-formato.sty`; en los capítulos no se definen macros. Las normas de redacción están en `apunte/ESTILO.md` y las vigila un script:

```
uv run python apunte/scripts/estilo.py --estricto
```

Lo que bloquea la publicación: «sólo» y demostrativos con tilde, comillas escritas a mano en vez de `\enquote` , punto decimal en el texto, «semejanza» aplicada a matrices, «es decir» más de una vez cada cien líneas, párrafos que empiezan con «Desarrollo.», etiquetas numéricas y definiciones repetidas. Lo demás (términos de campo, instrumentos, preguntas en «La idea») son advertencias.

5.2. Recompilar y commitear el PDF

Nadie compila dentro de `apunte/`. El único camino es el script protegido, desde la raíz:

```
scripts/compilar_pdf.sh
uv run python apunte/scripts/numeracion.py
git add apunte/Apunte_Sistemas_de_Coordenadas.pdf apunte/paginas_esperadas.txt
    ↪ apunte/numeracion.json
```

El script compila con LuaLaTeX en un directorio temporal y reemplaza el PDF solo si la compilación salió entera (sin referencias indefinidas ni archivos faltantes) y si pasa la verificación. Escribe también `paginas_esperadas.txt`, que va en el mismo commit que el PDF. El segundo comando regenera `numeracion.json`, que el sitio usa para enlazar secciones del apunte sin compilar LaTeX. Con `--sin-copiar` compila y verifica sin tocar nada; con `--destino RUTA` deja el PDF en otro lado.

Los ganchos de Git (sección 2) rechazan un commit cuyo PDF esté truncado, no coincida con `paginas_esperadas.txt` o tenga menos páginas que el commit anterior. Si la baja es legítima (se quitó una sección), el mensaje del commit lleva una línea `PAGINAS-BAJAN: motivo`.

5.3. Agregar un capítulo o un problema

1. Crear `apunte/contenido/NN-nombre.tex` (minúsculas, guiones, sin acentos) con un comentario de título, `\chapter {...}\label {cap:<clave>}` y las tres secciones del molde: «La idea», «Construcciones, cuentas y ejemplos» y «En profundidad». La etiqueta es un nombre, nunca un número; si ya está anunciada en `apunte/ETIQUETAS.md`, se usa ese nombre exacto.
2. Agregar `\input {contenido/NN-nombre}` en el maestro, en el lugar que le toca en el orden.
3. Si el capítulo lleva problemas graduados al final: agregar su `C[n] = ("Título", [...])` en `tools/problemas/gen.py`, donde cada problema es una tupla (nivel, enunciado, respuesta) con las cuentas en NumPy, y correr:
   ```
   python3 tools/problemas/gen.py --salida apunte/contenido
   ```
   Genera `contenido/problemas/pNN.tex` y las respuestas del apéndice; la línea `\input {contenido/problemas/pNN}` se agrega a mano en el maestro, después del capítulo.
4. Registrar el capítulo en la ficha y en el mapa (sección 6) con la clave de su etiqueta, sin el prefijo `cap:`.
5. Estilo, compilar, numeración y commit, como en las dos subsecciones anteriores.

Las figuras y los números del apunte salen siempre de un script en `apunte/scripts/`; las comprobaciones con código viven en `apunte/demostraciones/` y sus fragmentos se extraen a `apunte/generado/` al compilar. La publicación exige que esos fragmentos commiteados coincidan con las pruebas.

6. Fichas de capítulo y mapa de conceptos

6.1. Editar

Una ficha es `clases/NN-tema.md`: un frontmatter con `clase`, `titulo`, `idea`, `que_se_estudia`, `conceptos_previos`, `apunte` (claves de etiqueta sin `cap:`), `unidades` (identificadores), `animaciones` (nombres sin extensión) y `ejercicios_sugeridos`, seguido de las secciones «Qué se estudia», «Conceptos previos», «En el apunte», «Unidades interactivas», «Animaciones» (si hay) y «Ejercicios sugeridos». El molde está en `clases/PLANTILLA.md`. El generador del sitio valida el frontmatter y falla si una animación nombrada no existe.

Los conceptos del mapa viven en `mapa/_fragmentos/bloque-*.yaml`, uno por grupo de fichas. Cada concepto tiene identificador, nombre, ficha, unidad que lo desarrolla, hecho geométrico, qué se conserva, secciones del apunte y qué requiere. Editar un concepto es editar su entrada; el vocabulario de identificadores está en `mapa/CONCEPTOS.md`. Después de cualquier cambio:

```
uv run tools/merge_mapa.py        # valida y regenera mapa/mapa.yaml
uv run python tools/build.py --skip-wasm
```

y se commitea el `mapa.yaml` regenerado: la publicación compara el commiteado con los fragmentos y se detiene si difieren.

6.2. Agregar una ficha

1. Crear `clases/NN-tema.md` desde la plantilla.
2. Registrarla en la lista `clases:` del `bloque-*.yaml` que corresponda, con `id`, `titulo`, `ficha`, `pregunta` y `nivel`.
3. Darle rótulo visible en `CAPITULO_DE_FICHA` y `PROGRAMA_AVANCE`, al principio de `tools/build.py`; sin eso se muestra el identificador crudo. Ojo: el identificador de ficha sigue el orden interno y el rótulo usa la numeración del PDF (la ficha 03 cubre los capítulos 3 y 4; de la 04 a la 14 el capítulo es el identificador más uno; 03p es C1).
4. `merge_mapa.py`, `build.py`, commit.

7. Animaciones y laboratorio CAD

Animaciones. Cada una es un módulo `tools/animaciones/anim_NN_nombre.py` con una función `render(out_dir, s)` que escribe un cuadro PNG por valor de `s`, usando la paleta y los ayudantes de `_comun.py`. Para agregar una: crear el módulo, sumar su nombre a la lista `ANIMACIONES` de `tools/gen_animaciones.py`, generar y commitear el `.mp4` y el `.gif` que quedan en `animaciones/`, y nombrarla en el campo `animaciones:` de la ficha. La descripción que aparece en el menú es la primera oración del docstring del módulo.

```
uv run python tools/gen_animaciones.py --only 07     # una sola, por prefijo
uv run python tools/gen_animaciones.py --rapido      # 20 fps, para probar
uv run python tools/gen_animaciones.py               # las seis (o siete)
```

La ruta de `ffmpeg` está fija en el script (`/opt/homebrew/bin/ffmpeg`); en otra máquina hay que ajustarla.

CAD. Los ejercicios («cada comando es una transformación») tienen tres lugares que deben coincidir: el diccionario `EJERCICIOS` de `tools/cad/generar_dxf.py`, que produce los DXF; `cad/ejercicios.yaml`, que alimenta los menús (capítulo, archivo, título, comando, respuesta esperada); y la sección «Capítulo N» de `cad/index.md`, que es la página del sitio. Después de editar:

```
uv run python tools/cad/generar_dxf.py
uv run pytest tools/cad -q
uv run python tools/cad/que_matriz.py cad/dxf/cap01_rotate.dxf cad/dxf/esperado/
   ↪ cap01_rotate_30.dxf
```

El generador es determinista: la misma fuente produce los mismos bytes, así que un DXF que cambia sin que cambie el script es señal de edición a mano.

8. Verificar y publicar

La publicación la hace GitHub Actions con cada `push` a `main`: instala las dependencias congeladas, y corre en orden la fusión del mapa, la comparación del `mapa.yaml` commiteado, `marimo check` de las lecciones, el estilo del apunte y de lo interactivo, las pruebas de las demostraciones y del CAD, la coincidencia de los fragmentos generados, la verificación del PDF y de que sus páginas no bajen, la generación del sitio con exportación de las veinte unidades y la búsqueda de términos prohibidos sobre el artefacto. Cualquier paso que falla detiene la publicación y el sitio sigue mostrando la versión anterior. En un pull request corre lo mismo sin desplegar, así que abrir una PR es la forma segura de probar un cambio grande.

Lista previa a publicar, desde la raíz del repositorio

```
uv run tools/merge_mapa.py --check
uv run python tools/merge_banco.py --check
uv run marimo check --select MW unidades/[01]*.py
uv run python apunte/scripts/estilo.py --todo --estricto
uv run pytest -q
uv run python tools/build.py
uv run python apunte/scripts/prohibidas.py --dir site
git status --short         # lo generado que cambio tiene que ir en el commit
```

Después:

```
git add <fuentes editadas> <derivados regenerados>
git commit -m "Que cambio y por que"
git push
```

El progreso se sigue en la pestaña «Actions» del repositorio; el sitio queda actualizado en unos minutos.

8.1. Publicar bajo la cuenta de la cátedra

El repositorio está preparado para que la cátedra lo tome y lo publique con su propia firma, en cuatro pasos:

1. Hacer un *fork* desde GitHub (botón «Fork» en la página del repositorio) y clonarlo.
2. Cambiar la firma, primero en seco para ver qué tocaría y después en serio:
   ```
   uv run python tools/adoptar_para_catedra.py --dry-run --autor "Catedra
      ↪ Sistemas de Coordenadas (CB104) - Agrimensura FIUBA" --url-sitio "https
      ↪ ://<usuario>.github.io/Sistemas-de-Coordenadas/"
   uv run python tools/adoptar_para_catedra.py --autor "..." --url-sitio "..."
   ```

   Reemplaza la línea de firma de las veinte unidades, el pie de todas las páginas del sitio, la portada y los metadatos del apunte y la dirección del sitio en los documentos. No toca el contenido. Es idempotente.
3. Recompilar el apunte para que la portada muestre la firma nueva (sección 5); si no se hace, el sitio sirve el PDF con la firma original y todo lo demás ya queda con la nueva.
4. Commit, push, y una sola vez en GitHub: «Settings → Pages → Build and deployment → Source: GitHub Actions». Desde ahí, cada push a `main` publica en `https://<usuario>.github.io/Sistemas-de-Coordenadas/`.

9. Resumen: qué comando para cada tarea

Tarea	Comandos (desde la raíz)
Editar una unidad	`uv run marimo edit unidades/<id>.py`; verificar con `marimo check --select MW`, correr el `.py`, `build.py --only <id>`.
Agregar una unidad	Copiar `PLANTILLA_leccion.py`; alta en `unidades.yaml`; conceptos en `bloque-*.yaml`; `HILO_CONDUCTOR`; `enlaces_portada.yaml`; seis preguntas; `merge_mapa.py`; `build.py --only`.
Editar o agregar preguntas	Editar `unidades/_banco/bloque_N.py`; `merge_banco.py`; `build.py`; commitear el banco regenerado y el XML.
Editar el apunte	Editar `apunte/contenido/NN-nombre.tex`; `estilo.py --estricto`; `scripts/compilar_pdf.sh`; `numeracion.py`; commitear PDF, páginas esperadas y numeración.
Agregar un capítulo	Nuevo `NN-nombre.tex` con `\label {cap:<clave>}`; `\input` en el maestro; `C[n]` en `tools/problemas/gen.py`; ficha y mapa; compilar.
Editar ficha o mapa	Editar `clases/NN-tema.md` o `mapa/_fragmentos/*.yaml`; `merge_mapa.py`; `build.py --skip-wasm`; commitear `mapa.yaml`.
Agregar una animación	Módulo en `tools/animaciones/`; lista `ANIMACIONES`; `gen_animaciones.py --only NN`; commitear mp4 y gif; nombrarla en la ficha.
Editar el CAD	`EJERCICIOS` en `generar_dxf.py`, `cad/ejercicios.yaml`, `cad/index.md`; `generar_dxf.py`; `pytest tools/cad`.
Publicar	Lista previa de la sección 8; `git add`, `git commit`, `git push`; mirar «Actions».
Deshacer	`git checkout -- <archivo>` para cambios sin commitear; `git revert <commit>` para un commit ya publicado.

Los detalles que este instructivo resume están en los README de cada carpeta (`tools/README.md`, `apunte/README.md`, `clases/README.md`, `unidades/_banco/README.md`, `scripts/README.md`, `moodle/README.md`) y en `INSTRUCTIVO_DE_USO.md`. El texto, el apunte, las figuras y las preguntas se publican con licencia CC BY-SA 4.0; el código, con licencia MIT.